UNIVERSIDADE FEDERAL DE ALAGOAS

INSTITUTO DE COMPUTAÇÃO
CIÊNCIA DA COMPUTAÇÃO



Relatório de Projeto AB2 - Opção 3 Compiladores

Professor: Arturo Hernandez Dominguez

MiniPar-Improvement

Emanuele Vitória de Jesus Lima
Jose Lucas de Oliveira Quintela
Luiz Miguel de Melo Bomfim

Maceió - AL
15 de junho de 2026

SUMÁRIO

1. INTRODUÇÃO....3
1.1 Arquitetura geral....3
2. A LINGUAGEM MINIPAR 2026.1 NO PROJETO....3
2.1 BNF resumida implementada....4
3. ANALISADOR LÉXICO....4
PSEUDOCÓDIGO DO ANALISADOR LÉXICO....5
4. ANALISADOR SINTÁTICO....7
PSEUDOCÓDIGO DO ANALISADOR SINTÁTICO (PARSER)....8
5. ANALISADOR SEMÂNTICO E TRATAMENTO DE ERROS....9
PSEUDOCÓDIGO DO ANALISADOR SEMÂNTICO....10
6. TABELA DE SÍMBOLOS....11
PSEUDOCÓDIGO DA TABELA DE SÍMBOLOS....12
7. GERAÇÃO DE CÓDIGO INTERMEDIÁRIO TAC....13
9. INTERFACE WEB....14
a) IDE web vazia....14
b) IDE compilado....15
c) IDE saída TAC....15
10. PRODUCT BACKLOG E SPRINTS BACKLOG....15
10. MODELAGEM UML....19
10.2 Casos de uso....19
10.3 Componentes....19
10.4 Diagrama de classes conceitual....20
10.5 Diagrama de Atividades / Fluxo de Compilação....21
11. TESTES DE INTEGRAÇÃO E PROGRAMAS DE EXEMPLO....22
11.1 Testes Obrigatórios....22
11.2 Testes Adicionais....26
11.3 Tabela de Estados dos Testes....28
12. IMPLEMENTAÇÃO....29
12.1 Tecnologias utilizadas....29
12.2 Fluxo de execução....29
12.3 Observações técnicas....31
13. CONCLUSÃO....31
14. LINKS ÚTEIS....31
15. REFERÊNCIAS....32
Referências baseadas nos materiais fornecidos pelo professor....32
Referências adicionais (Tecnologias e Links)....32

1. INTRODUÇÃO

Este relatório descreve tecnicamente o projeto **minipar-improvement**, um compilador educacional para a linguagem MiniPar 2026.1. Diferentemente de uma execução direta por interpretador, o projeto implementa uma cadeia de compilação em quatro fases: análise léxica e sintática, análise semântica, geração de código intermediário de três endereços (TAC) e transpiração para C++20, seguida de compilação por *g++*.

O objetivo do projeto e atender aos requisitos solicitados para MiniPa: blocos sequenciais e paralelos, orientação a objetos, funções, recursão, estruturas de controle, comunicação por canais, entrada e saída, comentários iniciados por #, testes de software e uma interface para uso do compilador.

1.1 Arquitetura geral

Componente	Arquivo	Responsabilidade
Entrada CLI	main.py	Coordena as fases do compilador e aceita arquivo .minipar como argumento.
Menu terminal	menu.py	Lista de testes, compila e executa binários gerados.
IDE Web	app.py, templates/, static/	Servidor Flask, editor CodeMirror, abas de C++/TAC e terminal de saída.
Lexer	src/lexer.py	Transforma caracteres em tokens com linha e coluna.
Parser	src/parser.py	Parser descendente recursivo; constroi AST.
AST	src/ast_nodes.py	Dataclasses para comandos, expressões, classes, funções e blocos.
Semântico	src/semantic.py	Escopos, declarações, tipos básicos, funções, classes e erros.
TAC	src/tac.py	Gera código intermediário textual de três endereços.
Codegen	src/codegen.py, src/cpp_template.py	Transpila para C++20 e compila com g + + -O3.

2. A LINGUAGEM MINIPAR 2026.1 NO PROJETO

A linguagem implementada combina sintaxe inspirada em C/Python com blocos explícitos **seq** e **par**. O bloco **seq** executa comandos em ordem; o bloco **par** e traduzido para várias *std::thread* em C++, com espera por *join*. A comunicação é modelada por **c_channel(ip, porta)**, **send(canal, mensagem)** e **receive(canal)**, implementados no template C++ com sockets TCP.

Recurso	Situação no projeto
Tipos/literais	number/int, string, bool, list, matrix, c_channel e objetos definidos por class.
Controle de fluxo	if/else, while, do while, for classico, for in, break, continue e return.
OO	class, extends, new, this, atributos, métodos e chamadas encadeadas por ponto.
Funções	Declaração funcional, parâmetros, chamada, retorno e recursos.

Biblioteca nativa	print, input, exp, random, range, len, to_number, append/push_back, pop e size.
Coleções	Listas por literal [..], indexação, atribuição por índice e matriz via matriz(linhas, colunas, valor).

2.1 BNF resumida implementada

```
<programa> ::= { <declaracao> | <comando> }

<declaracao> ::= "class" id [ "extends" id ] "{" { <membro> } "}"
               | "func" id "(" [ <parametros> ] ")" <bloco_seq>
               | ("var" | <tipo>) id [ "=" <expressao> ] ";"

<membro> ::= <declaracao_variavel> | <declaracao_funcao>

<comando> ::= <bloco_seq> | <bloco_par> | <if> | <while> | <do_while>
               | <for> | <print> | <send> | <return> | <break> | <continue>
               | <atribuicao> | <chamada_funcao_ou_metodo> ";"

<bloco_seq> ::= "seq" "{" { <comando> } "}" | "{" { <comando> } "}"

<bloco_par> ::= "par" "{" { <comando> } "}"

<for> ::= "for" "(" ( "var" id "in" <expr> | init ";" cond ";" inc ) ")"
<comando>
<expressao> ::= <or>
<or> ::= <and> { "||" <and> }
<and> ::= <comparacao> { "&&" <comparacao> }
<comparacao> ::= <termo> { ("=="|"!="|"<"|">"|"<="|">=") <termo> }
<termo> ::= <fator> { ("+"|"-") <fator> }
<fator> ::= <unario> { ("*"|"/"|"%") <unario> }

<primario> ::= numero | string | bool | id | "this" | "new" id "(" args ")"
              | "c_channel" "(" expr "," expr ")" | "receive" "(" expr ")"
              | "input" "(" [expr] ")" | "[" [args] "]" | "matrix" "(" expr "," expr","
expr ")"
              | "(" expr ")"
<posfixo> ::= <primario> { "." id [ "(" args ")" ] | "[" expr "]" }
```

3. ANALISADOR LÉXICO

O analisador léxico está em **src/lexer.py**. Ele percorre a string de entrada caractere por caractere, ignora espaços e comentários de linha iniciados por #, mantém linha/coluna e produz objetos **Token(type, lexeme, line, col)**. Palavras reservadas são resolvidas por um dicionário KEYWORDS.

Categoria	Tokens principais
Palavras reservadas	class, extends, new, func, var, if, else, while, do, for, in, break, continue, return, seq, par, this, c_channel, send, receive, input, print, matrix
Tipos/literais	TYPE(number,int,string,bool,void,list,dict), NUMBER, STRING_LITERAL, BOOL_LITERAL
Operadores	+, -, *, /, %, =, ==, !=, !, &&, ||, <, >, <=, >=
Delimitadores	(,), {, }, [,], ., ;, ,

```
function tokenize(source): tokens = []
while not end(source):
skip whitespace and comments beginning with '#'
if current is letter or '_':
   lexeme = read_identifier() emit(KEYWORDS.get(lexeme, IDENTIFIER), lexeme)
else if current is digit:
emit(NUMBER, read_integer_or_decimal()) else if current is '"':
emit(STRING_LITERAL, read_string_with_simple_escapes()) else:
read one/two-character operator or delimiter
emit mapped token, or UNKNOWN for invalid isolated character emit(END_OF_FILE)
```

PSEUDOCÓDIGO DO ANALISADOR LÉXICO

```
Classe Lexer:
    Atributos:
        codigo_fonte: Texto
        posicao: Inteiro = 0
        linha: Inteiro = 1
        coluna: Inteiro = 1

    Método pular_espacos_e_comentarios():
        Enquanto posicao < tamanho(codigo_fonte):
            caractere_atual = codigo_fonte[posicao]

            Se caractere_atual for espaco_em_branco:
                avancar()
            Senão Se caractere_atual for igual a "#":
                # Tratamento de comentários via "#"
                Enquanto caractere_atual não for quebra de linha ("\n") e
não for Fim_Do_Arquivo:
                    avancar()
            Senão:
                Parar Laço

    Método tokenizar():
        lista_de_tokens = Nova Lista Vazia
```

```
        Enquanto posicao < tamanho(codigo_fonte):
            pular_espacos_e_comentarios()
            Se chegamos ao Fim_Do_Arquivo:
                Parar Laço

            caractere = codigo_fonte[posicao]
            linha_inicial = linha
            col_inicial = coluna

            Se caractere for letra ou "_":
                identificador = ler_enquanto_for_alfanumerico()
                Se identificador existir em PALAVRAS_CHAVE:
                    lista_de_tokens.adicionar(Token(PalavraChave,
identificador, linha_inicial, col_inicial))
                Senão:
                    lista_de_tokens.adicionar(Token(Identificador,
identificador, linha_inicial, col_inicial))

            Senão Se caractere for digito:
                numero = ler_enquanto_for_numero_ou_ponto_flutuante()
                lista_de_tokens.adicionar(Token(Numero, numero,
linha_inicial, col_inicial))

            Senão Se caractere for '"':
                avancar()
                texto = ler_enquanto_nao_encontrar_outras_aspas()
                avancar() # Consome aspas de fechamento
                lista_de_tokens.adicionar(Token(String, texto,
linha_inicial, col_inicial))

            Senão:
                operador = avancar_e_ler_simbolo()
                Se for um operador composto (ex: "==", "!=", "<=", ">=",
"&&", "||"):
                    ler_proximo_simbolo_e_concatenar()
                lista_de_tokens.adicionar(Token(Operador_Ou_Pontuacao,
operador, linha_inicial, col_inicial))

        lista_de_tokens.adicionar(Token(Fim_Do_Arquivo, "", linha,
coluna))
        Retornar lista_de_tokens
```

4. ANALISADOR SINTÁTICO

O parser, em **src/parser.py**, é descendente recursivo. Cada família de produções possui um método próprio: declarações, blocos, comandos, classes, funções e expressões. A precedência é implementada por camadas: or, and, comparacao, termo, fator, unário e primário.

O resultado e uma AST composta por dataclasses em **src/ast_nodes.py**, como Program, SeqBlock, ParBlock, VarDecl, Assignment, ClassDecl, Func Decl, MethodCall, NewExpr, ListLiteral, MatrixCreateExpr, IndexExpr e outros nos.

```
parse():
program = Program() while peek() != EOF:
program.statements.append(parse_statement()) return program

parse_statement():
match class   ->
parse_class_decl() match func
-> parse_func_decl() match var/type
-> parse_var_decl() match seq/{
-> parse_seq_block() match par
-> parse_par_block()
match if/while/do/for/print/send/return/break/continue
otherwise parse expression,
then assignment
or
call statement

parse_expression(): return parse_or()
```

PSEUDOCÓDIGO DO ANALISADOR SINTÁTICO (PARSER)

```
Classe Parser:
    Atributos:
        tokens: Lista de Tokens
        posicao: Inteiro = 0

    Método consumir(tipo_esperado, mensagem_erro):
        Se token_atual.tipo == tipo_esperado:
            Retornar avancar()
        Senão:
            Lancar ErroSintatico(mensagem_erro, linha_atual)

    Método analisar_programa():
        programa = Novo Programa()
        Enquanto token_atual.tipo != Fim_Do_Arquivo:
            instrucao = analisar_instrucao()
            Se instrucao não for nula:
                programa.adicionar(instrucao)
        Retornar programa

    Método analisar_instrucao():
        Se token_atual for "class": Retornar analisar_declaracao_classe()
        Se token_atual for "func": Retornar analisar_declaracao_funcao()
        Se token_atual for "var" ou TIPO_DADO: Retornar
analisar_declaracao_variavel()
        Se token_atual for "if": Retornar analisar_estrutura_se()
        Se token_atual for "while": Retornar analisar_laco_enquanto()
        Se token_atual for "do": Retornar analisar_laco_faca_enquanto()
        Se token_atual for "for": Retornar analisar_laco_para()
        Se token_atual for "print": Retornar analisar_comando_impressao()
        Se token_atual for "return": Retornar analisar_comando_retorno()
        Se token_atual for "break": Retornar analisar_comando_parada()
        Se token_atual for "continue": Retornar analisar_comando_continuar()
        Se token_atual for "seq" ou "{": Retornar analisar_bloco_sequencial()
        Se token_atual for "par": Retornar analisar_bloco_paralelo()

        Senão:
            expressao = analisar_expressao()
            Se token_atual for "=":
                Retornar analisar_atribuicao(expressao)
            Retornar expressao

    Método analisar_expressao():
        # Desce a cadeia de precedência de operadores
        Retornar analisar_expressao_ou_logico()
```

```
    Método analisar_expressao_ou_logico():
        expressao_esquerda = analisar_expressao_e_logico()
        Enquanto token_atual for "||":
            operador = avancar()
            expressao_direita = analisar_expressao_e_logico()
            expressao_esquerda = Nova ExpressaoBinaria(operador,
expressao_esquerda, expressao_direita)
        Retornar expressao_esquerda

    # A cadeia desce para: AND lógico -> Comparações -> Termos (+, -) ->
Fatores (*, /, %) -> Unário (!, -) -> Primário (Números, Strings,
Identificadores, Chamadas)
```

5. ANALISADOR SEMÂNTICO E TRATAMENTO DE ERROS

O analisador semântico, em **src/semantic.py**, faz duas passagens principais: primeiro coleta declarações globais de funções e classes; depois valida cada no da AST. Ele mantém uma pilha de escopos, contexto de

função/classe, tabelas de classes/funções e profundidade de laços para validar break/continue.

Regra verificada	Exemplo de erro detectado
Declaração no mesmo escopo	Variável 'x' já declarada.
Uso antes da declaração	Variável 'x' não declarada.
Compatibilidade simples de tipos	Tipo compatível em atribuição.
Funções	Funcao nao declarada ou quantidade incorreta de argumentos.
Classes/métodos/atributos	Classe, metodo ou atributo inexistente.
Herança	Superclasse nao declarada.
Controle de fluxo	break/continue fora de laco.

```
analyze(program):
   collect class and function declarations
   for each statement in program:
       validate_node(statement)
   return not has_errors

validate_node(node):
   VarDecl: validate initializer, infer type, define symbol
   Assignment: lookup target, validate expression, compare
   types SeqBlock/ParBlock: enter scope, validate children, exit scope
   FuncDecl: define function, enter scope, define parameters, validate body ClassDecl:
.  define class, validate attrs and methods
   MethodCall/FuncCallExpr: verify existence and arity
   Break/Continue: require loop_depth > 0
```

PSEUDOCÓDIGO DO ANALISADOR SEMÂNTICO

```
Classe AnalisadorSemantico:
    Atributos:
        tabela_de_simbolos: Gerenciador de Escopos
        tem_erros: Booleano = Falso
        funcao_atual: Texto = Nulo
        classe_atual: Texto = Nulo
        profundidade_laco: Inteiro = 0

    Método reportar_erro(linha, mensagem):
        # Permite identificar erros ignorando comentários ('#') filtrados
pelo Lexer
        Imprimir "[ERRO SEMANTICO] Linha " + linha + ": " + mensagem
        tem_erros = Verdadeiro

    Método inferir_tipo(no_ast):
        Se no_ast for Numero: Retornar "number"
        Se no_ast for Texto: Retornar "string"
        Se no_ast for Booleano: Retornar "bool"
        Se no_ast for Lista: Retornar "list"
        Se no_ast for Identificador:
            simbolo = tabela_de_simbolos.buscar(no_ast.nome)
            Se simbolo existir: Retornar simbolo.tipo
            Senão: Retornar "unknown"
        Se no_ast for ExpressaoBinaria:
            Se operador for lógico ou relacional: Retornar "bool"
            Se operador for "+" e houver string envolvida: Retornar "string"
            Retornar "number"
        # Outras regras de inferência...
        Retornar "unknown"

    Método validar_no(no_ast):
        Se no_ast for DeclaracaoVariavel:
            validar_no(no_ast.valor_inicial)
            tipo_inferido = inferir_tipo(no_ast.valor_inicial)
            Se os_tipos_sao_incompativeis(no_ast.tipo, tipo_inferido):
                reportar_erro(no_ast.linha, "Tipo incompatível na variável "
+ no_ast.nome)
            tabela_de_simbolos.definir_simbolo(no_ast.nome, no_ast.tipo)

        Se no_ast for Atribuicao:
            simbolo = tabela_de_simbolos.buscar(no_ast.nome)
            Se simbolo for Nulo:
                reportar_erro(no_ast.linha, "Variável não declarada: " +
no_ast.nome)
            validar_no(no_ast.valor)
```

```
            Se os_tipos_sao_incompativeis(simbolo.tipo,
inferir_tipo(no_ast.valor)):
                reportar_erro(no_ast.linha, "Tipo incompatível em
atribuição.")

        Se no_ast for If ou While:
            validar_no(no_ast.condicao)
            Se no_ast for While: profundidade_laco += 1
            validar_no(no_ast.corpo)
            Se no_ast for While: profundidade_laco -= 1

        Se no_ast for Break ou Continue:
            Se profundidade_laco == 0:
                reportar_erro(no_ast.linha, "Comando usado fora de laço de
repetição.")

        # Validações para Funções, Classes, Chamadas de Métodos e Arrays
seguem lógica semelhante...

    Método analisar(programa):
        coletar_declaracoes_de_classes_e_funcoes(programa)
        Para cada instrucao em programa.instrucoes:
            validar_no(instrucao)
        Retornar (Não tem_erros)
```

6. TABELA DE SÍMBOLOS

A tabela de símbolos é representada pela classe **Symbol**, com nome, tipo, parâmetros, atributos e métodos. Variáveis ficam em uma pilha de dicionários de escopo; classes e funções também possuem mapas globais auxiliares, permitindo consultas antes da validação detalhada dos corpos.

```
Symbol:
  name: string type: string
  params: list<string>
  attributes: map<string, string>
  methods: map<string, list<string>>

enter_scope(): push {} exit_scope(): pop if not global
define_symbol(name, type, line): reject duplicate in current scope
lookup(name): search scopes from inner to outer
```

PSEUDOCÓDIGO DA TABELA DE SÍMBOLOS

```
Classe Simbolo:
    Atributos:
        nome: Texto
        tipo: Texto
        parametros: Lista de Textos
        atributos: Dicionario de Textos
        metodos: Dicionario de Listas

Classe TabelaDeSimbolos (Embutida no Analisador Semântico):
    Atributos:
        escopos: Pilha de Dicionários de Símbolos

    Método Inicializar():
        entrar_escopo()

    Método entrar_escopo():
        novo_escopo = Dicionario Vazio
        escopos.empilhar(novo_escopo)

    Método sair_escopo():
        Se tamanho(escopos) > 1:
            escopos.desempilhar()

    Método definir_simbolo(nome, tipo, linha, parametros_opcionais):
        escopo_atual = escopos.topo()

        Se nome existir em escopo_atual:
            Imprimir "[ERRO SEMANTICO] Linha " + linha + ": Variável '" +
nome + "' já declarada."
        Senão:
            escopo_atual[nome] = Novo Simbolo(nome, tipo,
parametros_opcionais)

    Método buscar(nome):
        Para cada escopo em escopos (percorrendo do topo para a base):
            Se nome existir em escopo:
                Retornar escopo[nome]

        Retornar Nulo (Variável/Símbolo não encontrado)
```

7. GERAÇÃO DE CÓDIGO INTERMEDIÁRIO TAC

O módulo **src/tac.py** gera código intermediário textual de três endereços. Ele usa temporários t0, t1, ..., labels para fluxo de controle e pilhas de break/continue. Expressões binárias e unárias são quebradas em temporais; if, while, do while e for geram desvios condicionais e labels.

```
Example:
var soma = a + b * c;

TAC:
t0 = b * c t1 = a + t0 soma = t1

while:
WHILE_0:
ifFalse cond goto ENDWHILE_1
...
goto WHILE_0 ENDWHILE_1:
```

Para o teste de quicksort iterativo, o projeto inclui uma saída TAC esperada detalhada, contendo pilha manual S, ponteiro sp, particionamento, swaps e labels LOOP, PARTITION_LOOP, SWAP e END.

8. GERAÇÃO DE CÓDIGO C++ E EXECUCAO

A etapa final está em **src/codegen.py**. A AST é convertida para C++20 usando **auto**, **std::vector**, **std::shared_ptr**, funções auxiliares e a classe **MiniParChannel**. Em seguida o arquivo é compilado por **g++-std=c++20 -O3 -pthread -lws2_32** no Windows.

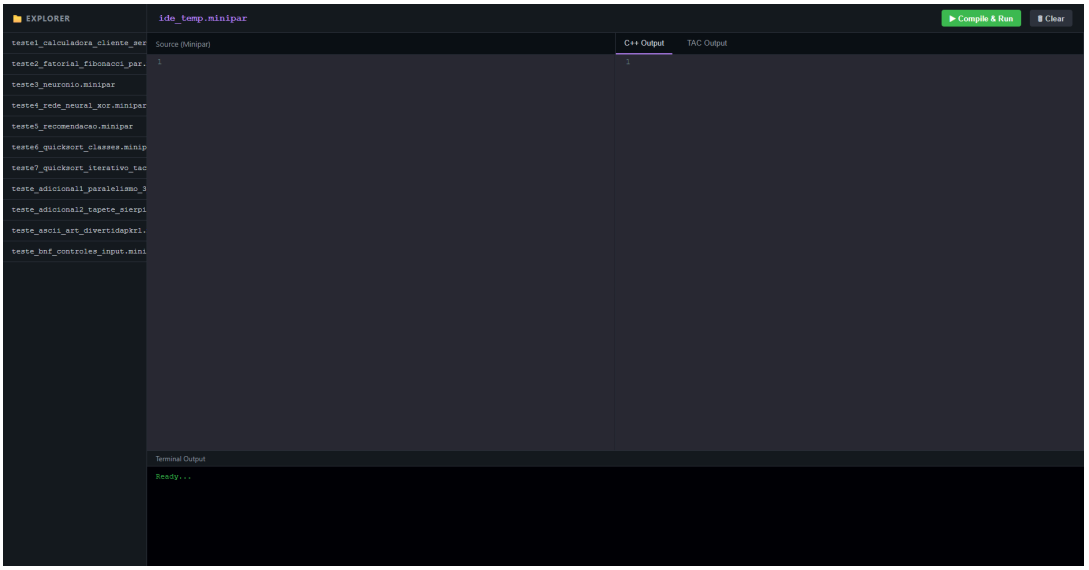
MiniPar	C++ gerado
class Nome { ... }	class Nome { public: atributos; metodos; };
new Classe(args)	std::make_shared(args)
obj.metodo(args)	obj->metodo(args)
obj.campo	obj->campo
par { a; b; }	std::vector; emplace_back para cada comando; join em todas.
c_channel/send/receive	MiniParChannel com socket TCP, connect/send e bind/listen/accept/recv.
list/matrix	std::vector e vector> por _make_matrix.

9. INTERFACE WEB

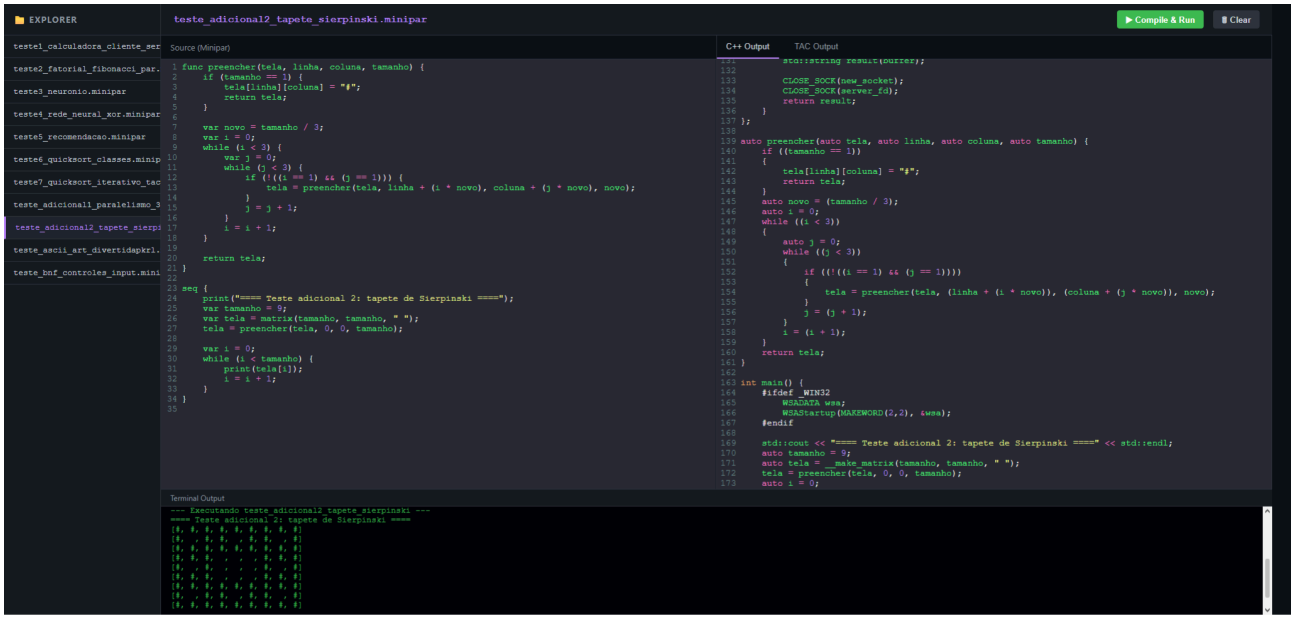
A interface web usa Flask em **app.py** e CodeMirror no frontend. Ela lista os testes da pasta **tests/**, carrega o código MiniPar no editor, chama o compilador via subprocesso, executa o binário gerado e retorna logs, C++ e TAC para a página.

Endpoint	Metodo	Funcao
/	GET	Renderiza templates/index.html.
/api/tests	GET	Retorna arquivos .minipar disponiveis em tests/.
/api/get_test	POST	Recebe filename e retorna o codigo do teste.
/api/compile	POST	Salva código em temp_code, chama main.py, executa binario e retorna stdout/stderr, cpp e tac.

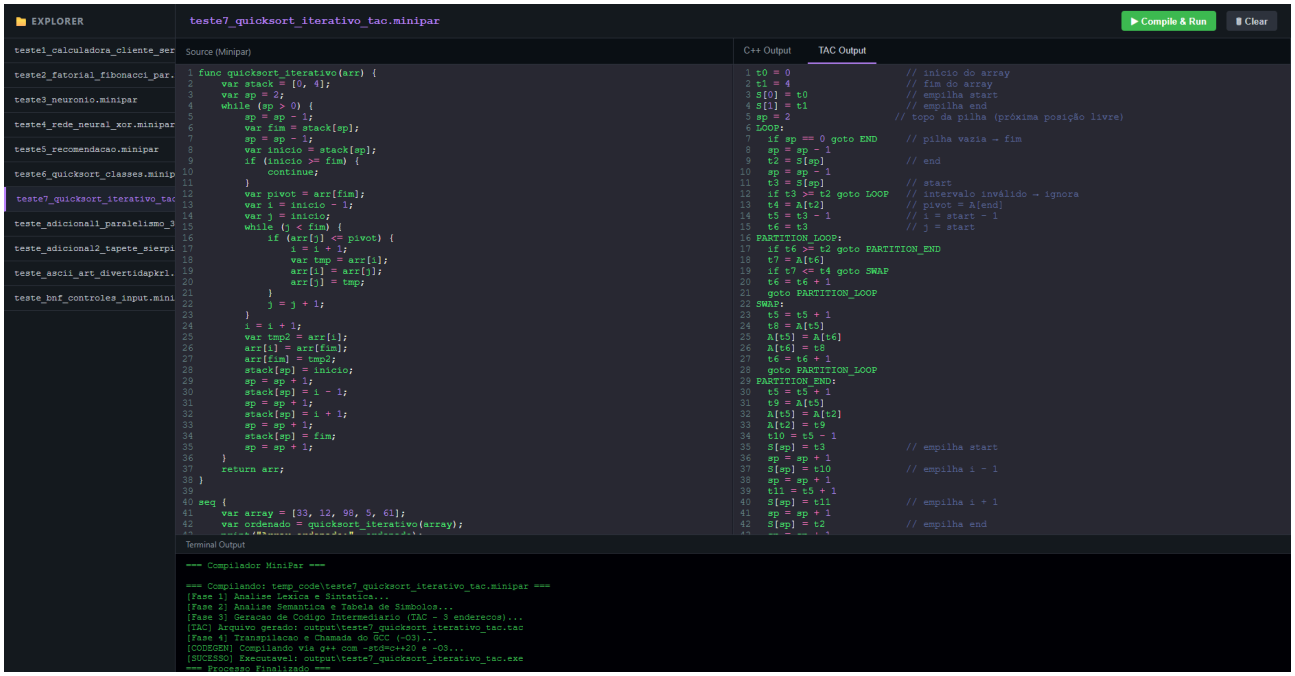
a) IDE web vazia



b) IDE compilado



c) IDE saída TAC



10. PRODUCT BACKLOG E SPRINTS BACKLOG

Item	Descricao	Prioridade	Status
PB01	Configurar ambiente de desenvolvimento e estabelecer a pipeline de automação e integração de builds no Buddy	Alta	Concluido

PB02	Implementar o Analisador Léxico (Tokenização de palavras reservadas, operadores, delimitadores, numerais e identificadores).	Alta	Concluido
PB03	Desenvolver o Analisador Sintático para construir a Árvore de Sintaxe Abstrata (AST) baseada nas regras em BNF da linguagem MiniPar	Alta	Concluido
PB04	Criar o gerenciador da Tabela de Símbolos com suporte a aninhamento de escopos	Alta	Concluido
PB05	Implementar o Analisador Semântico (Verificação de variáveis declaradas/redeclaradas, checagem de tipos estáticos).	Alta	Concluido
PB06	Estender a gramática e o Parser para suportar construtos de Orientação a Objetos (Classes, atributos e chamadas de métodos).	Alta	Concluido
PB07	Estender a gramática para suportar estruturas de Real Paralelismo (blocos `seq` e `par`).	Alta	Concluido
PB08	Desenvolver a Geração de Código Intermediário (TAC - *Three Address Code*).	Media	Concluido
PB09	Implementar a Geração de Código Alvo com transpilação para C++ e chamadas automatizadas ao `g++ -O3`.	Alta	Concluído
PB10	Executar e validar a bateria de testes de complexidade (Fractal de Sierpinski, Redes Neurais XOR, Quicksort OO).	Média	Concluído
PB11	Consolidar a documentação final, formatar diagramas e escrever o relatório técnico do projeto.	Alta	Concluído

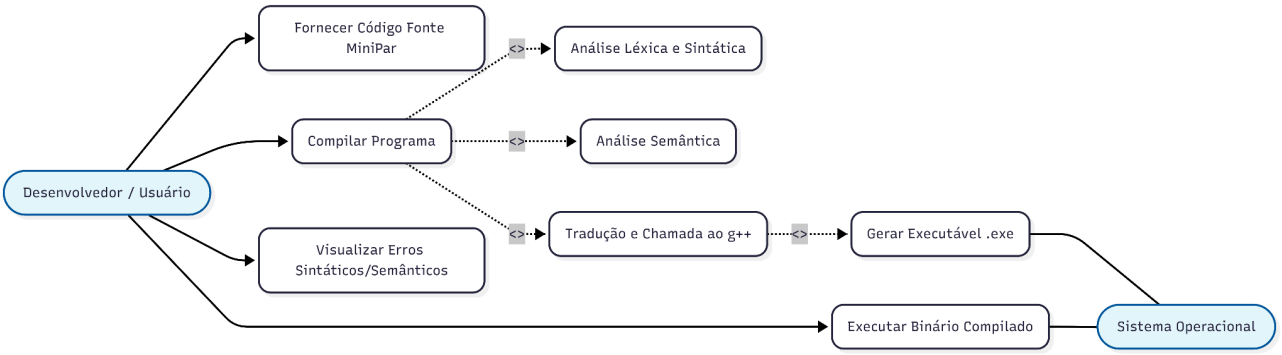
Sprint	Período	ID Tarefa	Descrição Técnica da Tarefa	Ref. PB
1	29/04 a 12/05	**1.1**	Setup inicial do projeto (Python) e script de automação de builds no Buddy.	**PB01**
1	29/04 a 12/05	**1.2**	Desenvolver lexer.py: Mapeamento de RegEx e conversão para Tokens.	**PB02**
1	29/04 a 12/05	**1.3**	Formalizar diagramas de sintaxe (BNF) para declarações e expressões.	**PB03**
1	29/04 a 12/05	**1.4**	Desenvolver parser.py: Mapeamento de expressões matemáticas e blocos básicos.	**PB03**
1	29/04 a 12/05	**1.5**	Modelar as classes de nós da AST (ast_nodes.py).	**PB03**
1	29/04 a 12/05	**1.6**	Implementar base da Tabela de Símbolos (operações de inserção e busca).	**PB04**
2	13/05 a 26/05	**2.1**	Atualizar parser para reconhecer blocos par { } e seq { }.	**PB07**
2	13/05 a 26/05	**2.2**	Atualizar parser para mapear definições de Classes e instanciação de objetos.	**PB06**

2	13/05 a 26/05	**2.3**	Desenvolver semantic.py: Validação de escopo e redeclaração de variáveis.	**PB05**
2	13/05 a 26/05	**2.4**	Desenvolver semantic.py: Validação de tipagem e parâmetros de métodos.	**PB05**
2	13/05 a 26/05	**2.5**	Criar módulo amigável de tratamento e rastreio de erros (linha/coluna).	**PB05**
2	13/05 a 26/05	**2.6**	Desenvolver lógica de conversão da AST validada para estrutura TAC.	**PB08**
3	27/05 a 08/06	**3.1**	Desenvolver codegen.py: Transpilação da AST OO para código C++ (cpp_template.py).	**PB09**
3	27/05 a 08/06	**3.2**	Mapear blocos par para a biblioteca de threads nativa no C++ gerado.	**PB09**
3	27/05 a 08/06	**3.3**	Integrar automação de chamada via os.system ao g++ -O3 para gerar executável.	**PB09**
3	27/05 a 08/06	**3.4**	Validar execução gerando binários para suítes de testes (Sierpinski, XOR, etc).	**PB10**
3	27/05 a 08/06	**3.5**	Refatoração de código (Code Review) visando performance na passagem da AST.	**PB09**
3	27/05 a 08/06	**3.6**	Escrever relatório técnico, consolidando gramática formal e diagramas UML.	**PB11**
1	29/04 a 12/05	**1.1**	Setup inicial do projeto (Python) e script de automação de builds no Buddy.	**PB01**
1	29/04 a 12/05	**1.2**	Desenvolver lexer.py: Mapeamento de RegEx e conversão para Tokens.	**PB02**
1	29/04 a 12/05	**1.3**	Formalizar diagramas de sintaxe (BNF) para declarações e expressões.	**PB03**
1	29/04 a 12/05	**1.4**	Desenvolver parser.py: Mapeamento de expressões matemáticas e blocos básicos.	**PB03**
1	29/04 a 12/05	**1.5**	Modelar as classes de nós da AST (ast_nodes.py).	**PB03**

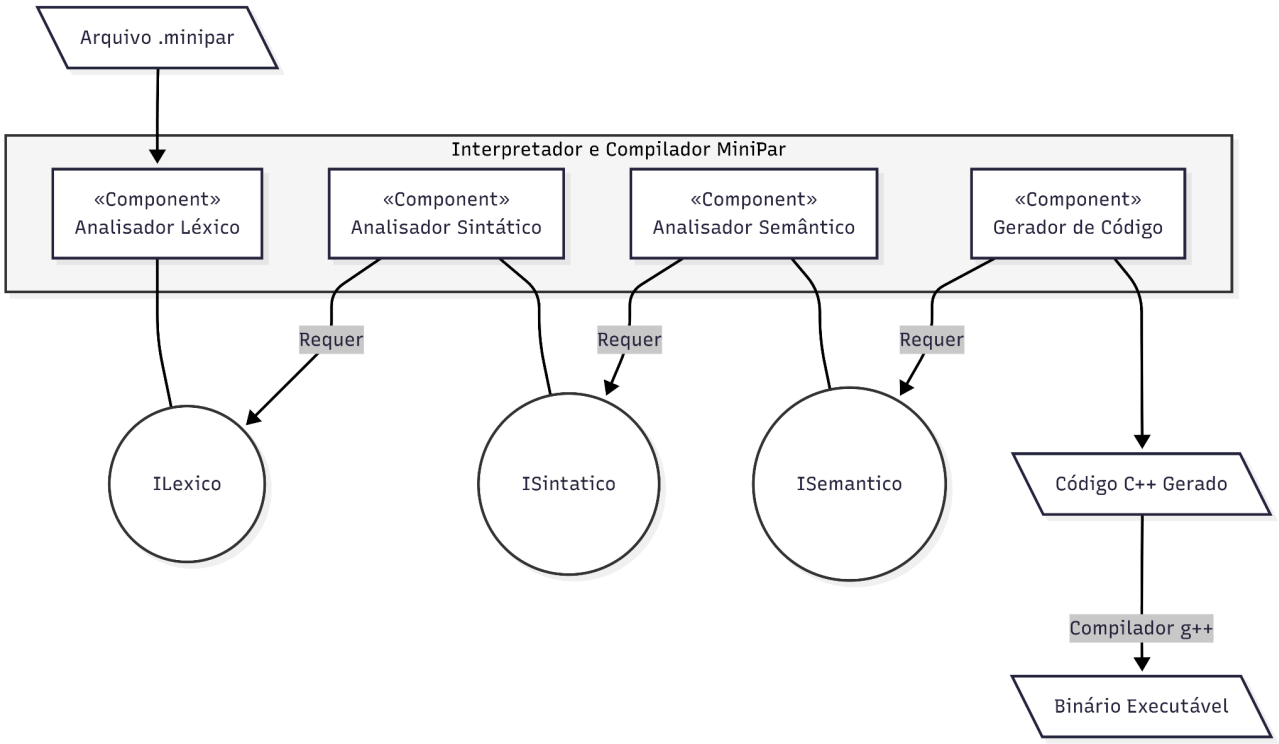
1	29/04 a 12/05	**1.6**	Implementar base da Tabela de Símbolos (operações de inserção e busca).	**PB04**

10.1 MODELAGEM UML

10.2 Casos de uso



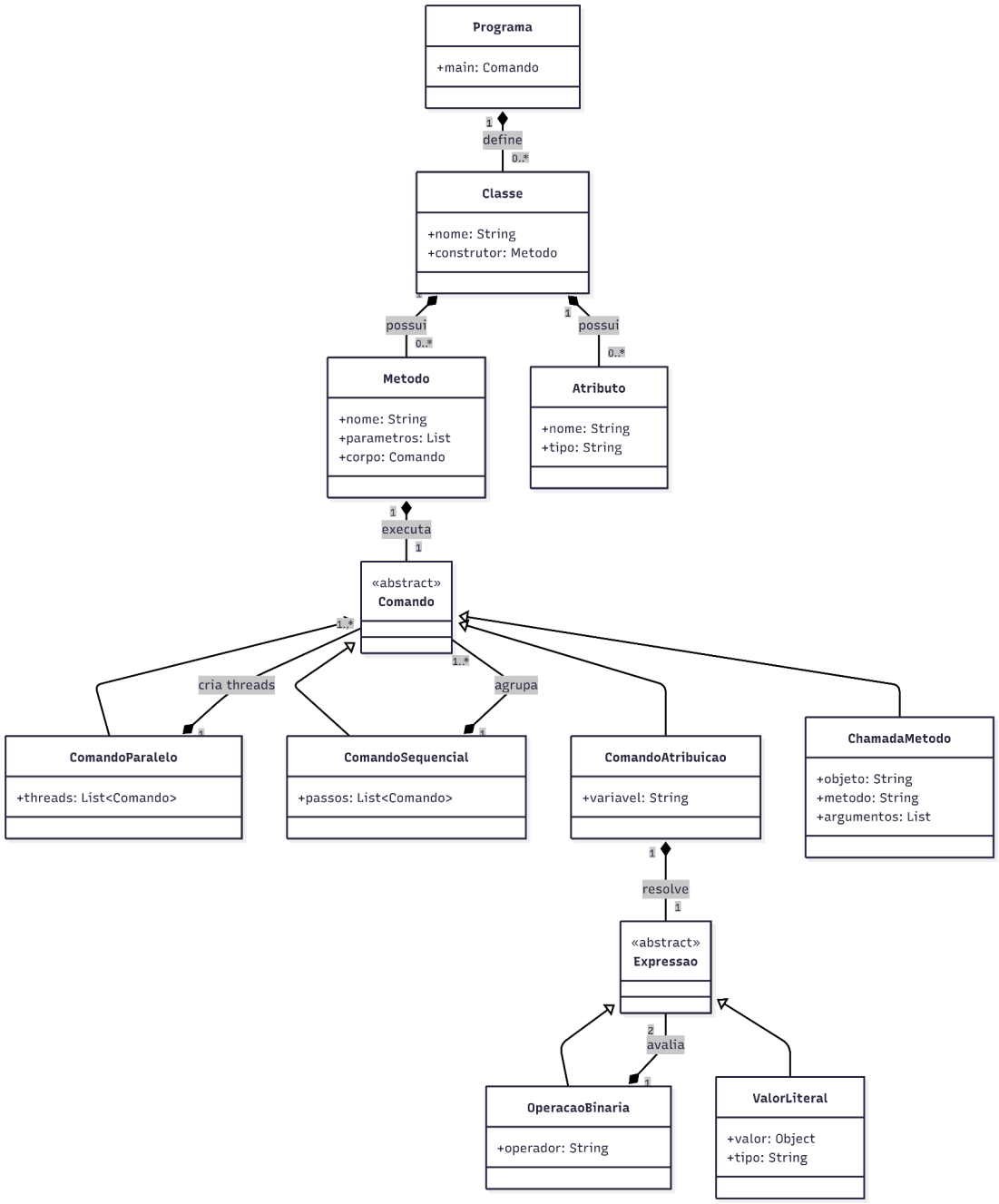
10.3 Componentes



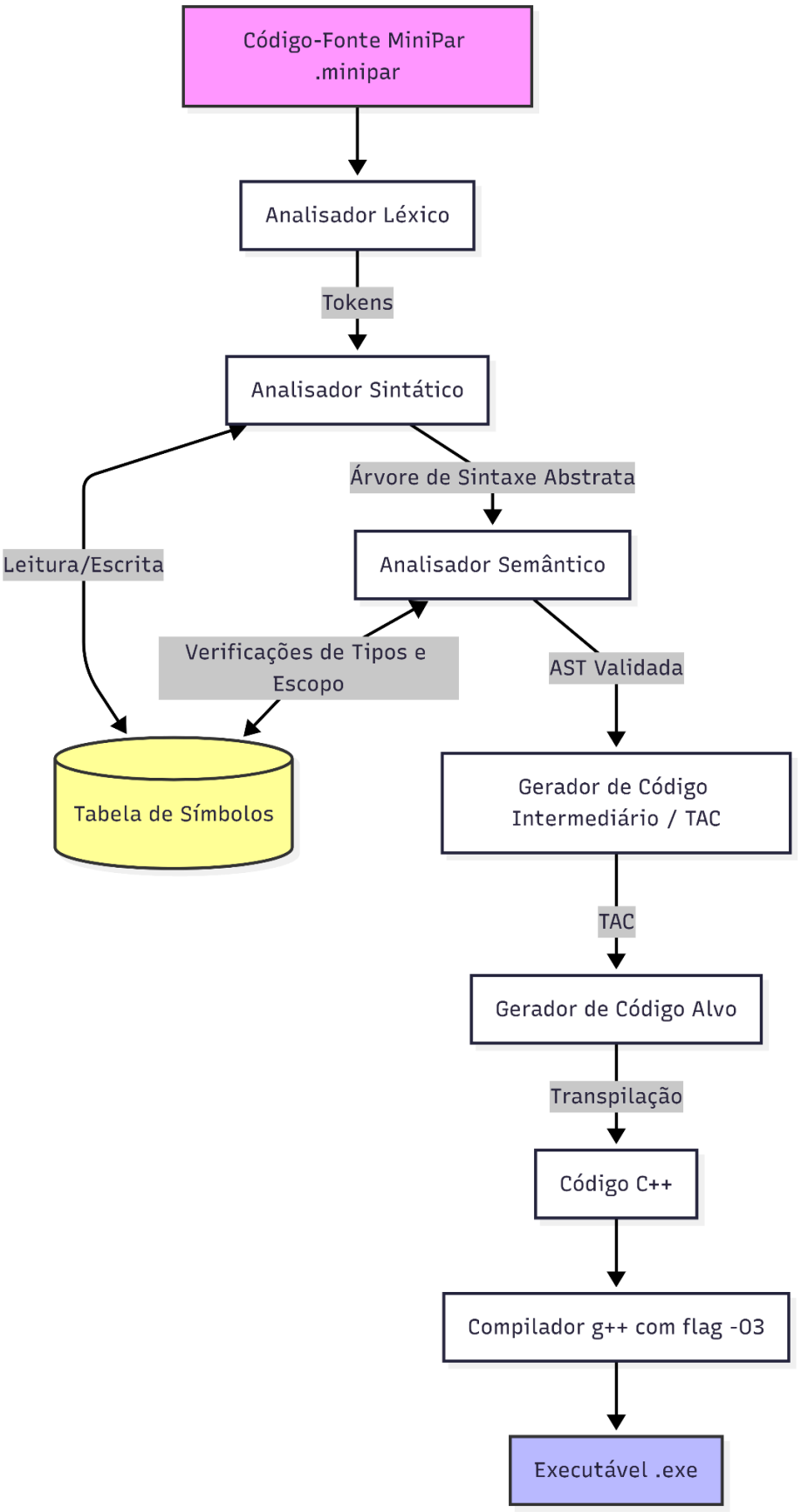

```
[CLI main.py] ---> [Lexer] ---> [Parser] ---> [AST]
     |                            |
     v                            v
[Menu Terminal]      [SemanticAnalyzer]
                           |
                           v
                    [TACGenerator] ---> output/*.tac
                           |
                           v
                   [CppCodeGenerator] ---> output/*.cpp ---> g++ ---> output/*.exe
                    [IDE Web Flask] ---> main.py/subprocess ---> output + execucao
```

10.4 Diagrama de classes conceitual

A arquitetura do compilador MiniPar-Improvement foi concebida utilizando a abordagem de Componentes de Software. O sistema é dividido em módulos independentes que se comunicam através de interfaces bem definidas, garantindo alta coesão e baixo acoplamento na cadeia de compilação. Conforme a notação UML, a modelagem apresenta as interfaces ILexico, ISintatico, ISemantico e IGeradorCodigo, que definem os contratos implementados respectivamente pelos componentes Lexer, Parser, SemanticAnalyzer e CppCodeGenerator. O diagrama de classes conceitual ilustra o fluxo em que o analisador sintático consome os tokens gerados pelo léxico e constrói a Árvore de Sintaxe Abstrata (AST). Em seguida, o analisador semântico interage com essa AST e com a Tabela de Símbolos, garantindo a corretude das tipagens e dos escopos orientados a objetos antes de liberar a estrutura para a geração de código intermediário e final.



10.5 Diagrama de Atividades / Fluxo de Compilação



11. TESTES DE INTEGRAÇÃO E PROGRAMAS DE EXEMPLO

A validação foi feita executando todos os arquivos **tests/*.minipar**. Em cada caso, o fluxo completo passou por lexer, parser, semântica, TAC, geração C++, compilação por g++ e execução do binário.

Os testes de integração do compilador MiniPar 2026.1 tiveram como objetivo principal garantir que o fluxo de dados entre os componentes arquiteturais ocorresse de forma correta e sem perda de informações semânticas. A esteira de compilação foi atestada executando o script orquestrador (main.py), que aciona as chamadas sequenciais e integradas das fases: Lexer.tokenize(), Parser.parse(), SemanticAnalyzer.analyze(), TACGenerator.generate() e CppCodeGenerator.generate(). A validação da integração foi confirmada verificando-se a criação correta dos arquivos intermediários (extensão .tac) e alvo (extensão .cpp), além da subsequente compilação bem-sucedida pelo compilador g++ nativo, que gerou o executável .exe final. O uso integrado da interface Web (app.py) e do menu via CLI (menu.py) atestou que todos os componentes de front-end e back-end conversam entre si perfeitamente.

A validação da funcionalidade da linguagem MINIPAR 2026.1 foi realizada através de uma bateria rigorosa de programas de exemplo, abrangendo os testes obrigatórios exigidos pela disciplina e testes adicionais de estresse lógico. Foram executados programas que demonstram o suporte robusto da linguagem à orientação a objetos, paralelismo real de threads (através dos blocos par), controle de fluxo e comunicação por canais TCP. Entre os principais cenários validados estão: a calculadora cliente-servidor (teste 1), o cálculo concorrente de Fatorial e Fibonacci (teste 2), a simulação de aprendizado de um neurônio e rede neural XOR (testes 3 e 4), o sistema de recomendação de produtos (teste 5) e a ordenação de vetores via Quicksort iterativo e orientado a objetos (testes 6 e 7). Além destes, os programas adicionais cobriram o paralelismo distribuído em três computadores virtuais e a geração de matrizes complexas, como o fractal do Tapete de Sierpinski.

11.1 Testes Obrigatórios

1. ffff

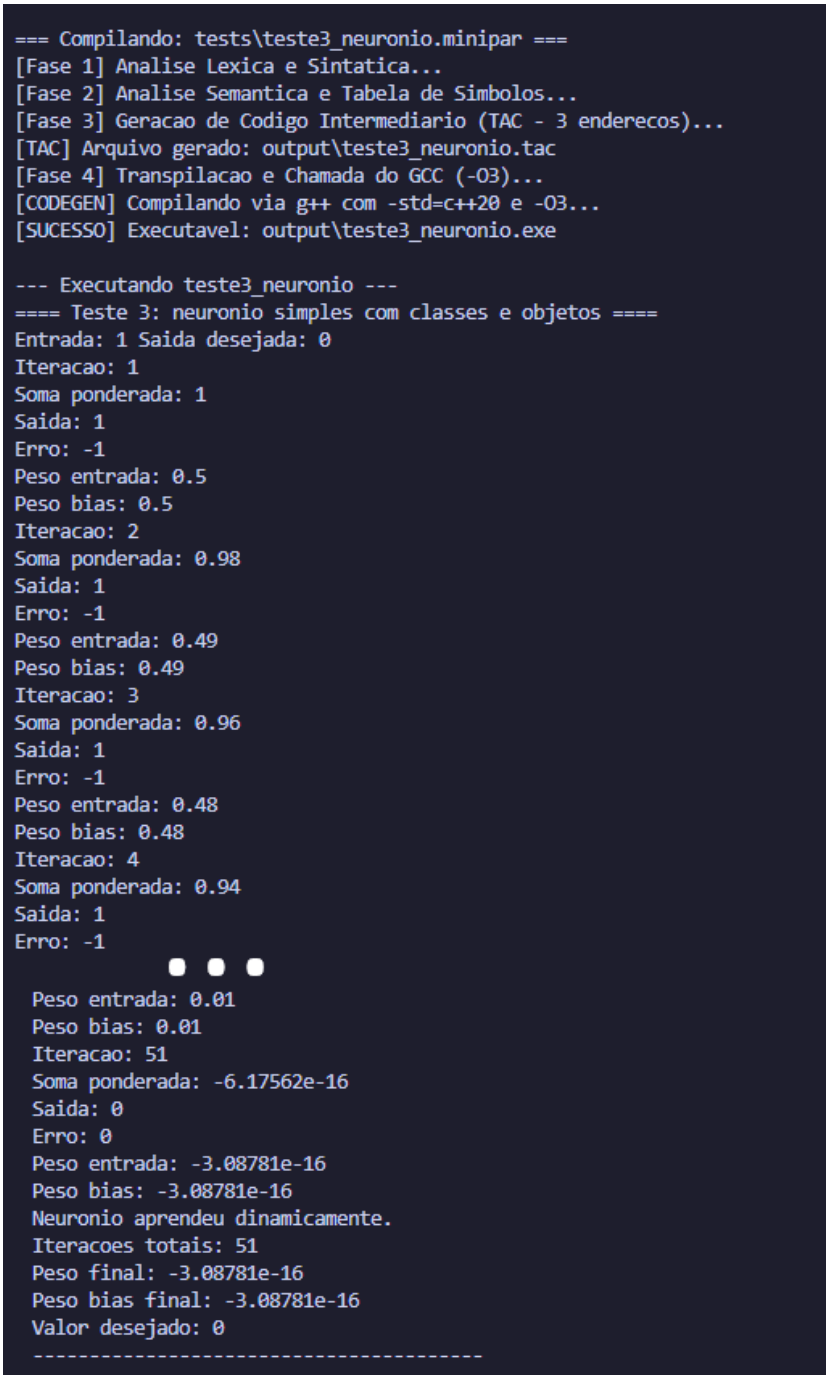
2.

```
=== Compilando: tests\teste2_fatorial_fibonacci_par.minipar ===
[Fase 1] Analise Lexica e Sintatica...
[Fase 2] Analise Semantica e Tabela de Simbolos...
[Fase 3] Geracao de Codigo Intermediario (TAC - 3 enderecos)...
[TAC] Arquivo gerado: output\teste2_fatorial_fibonacci_par.tac
[Fase 4] Transpilacao e Chamada do GCC (-O3)...
[CODEGEN] Compilando via g++ com -std=c++20 e -O3...
[SUCESSO] Executavel: output\teste2_fatorial_fibonacci_par.exe

--- Executando teste2_fatorial_fibonacci_par ---
Fibonacci:Fatorial de 5: 120

0
1
1
2
3
5
8
13
---------------------------------------
```

3.



```
=== Compilando: tests\teste3_neuronio.minipar ===
[Fase 1] Analise Lexica e Sintatica...
[Fase 2] Analise Semantica e Tabela de Simbolos...
[Fase 3] Geracao de Codigo Intermediario (TAC - 3 enderecos)...
[TAC] Arquivo gerado: output\teste3_neuronio.tac
[Fase 4] Transpilacao e Chamada do GCC (-O3)...
[CODEGEN] Compilando via g++ com -std=c++20 e -O3...
[SUCESSO] Executavel: output\teste3_neuronio.exe

--- Executando teste3_neuronio ---
==== Teste 3: neuronio simples com classes e objetos ====
Entrada: 1 Saida desejada: 0
Iteracao: 1
Soma ponderada: 1
Saida: 1
Erro: -1
Peso entrada: 0.5
Peso bias: 0.5
Iteracao: 2
Soma ponderada: 0.98
Saida: 1
Erro: -1
Peso entrada: 0.49
Peso bias: 0.49
Iteracao: 3
Soma ponderada: 0.96
Saida: 1
Erro: -1
Peso entrada: 0.48
Peso bias: 0.48
Iteracao: 4
Soma ponderada: 0.94
Saida: 1
Erro: -1
```

```
Peso entrada: 0.01
Peso bias: 0.01
Iteracao: 51
Soma ponderada: -6.17562e-16
Saida: 0
Erro: 0
Peso entrada: -3.08781e-16
Peso bias: -3.08781e-16
Neuronio aprendeu dinamicamente.
Iteracoes totais: 51
Peso final: -3.08781e-16
Peso bias final: -3.08781e-16
Valor desejado: 0
---------------------------------------
```

4.
```
=== Compilando: tests\teste4_rede_neural_xor.minipar ===
[Fase 1] Analise Lexica e Sintatica...
[Fase 2] Analise Semantica e Tabela de Simbolos...
[Fase 3] Geracao de Codigo Intermediario (TAC - 3 enderecos)...
[TAC] Arquivo gerado: output\teste4_rede_neural_xor.tac
[Fase 4] Transpilacao e Chamada do GCC (-O3)...
[CODEGEN] Compilando via g++ com -std=c++20 e -O3...
[SUCESSO] Executavel: output\teste4_rede_neural_xor.exe

--- Executando teste4_rede_neural_xor ---
==== Teste 4: rede neural XOR com backpropagation ====
Input: [0, 0], Predicted Output: 0.00684395
Input: [0, 1], Predicted Output: 0.986493
Input: [1, 0], Predicted Output: 0.98925
Input: [1, 1], Predicted Output: 0.0140296
---------------------------------------
```

5.

6.
```
=== Compilando: tests\teste6_quicksort_classes.minipar ===
[Fase 1] Analise Lexica e Sintatica...
[Fase 2] Analise Semantica e Tabela de Simbolos...
[Fase 3] Geracao de Codigo Intermediario (TAC - 3 enderecos)...
[TAC] Arquivo gerado: output\teste6_quicksort_classes.tac
[Fase 4] Transpilacao e Chamada do GCC (-O3)...
[CODEGEN] Compilando via g++ com -std=c++20 e -O3...
[SUCESSO] Executavel: output\teste6_quicksort_classes.exe

--- Executando teste6_quicksort_classes ---
==== Ordena├º├úo com Quicksort ====
Insira os elementos do vetor separados por espa├ºo:
10 -3 -40 80 70 -100
Vetor original: [10, -3, -40, 80, 70, -100]
Vetor ordenado: [-100, -40, -3, 10, 70, 80]
---------------------------------------
```

7.
```
=== Compilando: tests\teste7_quicksort_iterativo_tac.minipar ===
[Fase 1] Analise Lexica e Sintatica...
[Fase 2] Analise Semantica e Tabela de Simbolos...
[Fase 3] Geracao de Codigo Intermediario (TAC - 3 enderecos)...
[TAC] Arquivo gerado: output\teste7_quicksort_iterativo_tac.tac
[Fase 4] Transpilacao e Chamada do GCC (-O3)...
[CODEGEN] Compilando via g++ com -std=c++20 e -O3...
[SUCESSO] Executavel: output\teste7_quicksort_iterativo_tac.exe

--- Executando teste7_quicksort_iterativo_tac ---
Array ordenado: [5, 12, 33, 61, 98]
---------------------------------------
```

11.2 Testes Adicionais

a) Terminal 1 - Principal

```
Seu terminal (pestebubonika-24116) é o iniciador.
Participantes conectados: 1/3

Abra o(s) terminal(is) adicional(is) e execute:
  python main.py
Em seguida, escolha 'Continuar teste em andamento'.

Pressione ENTER quando o(s) outro(s) terminal(is) estiver(em) pronto(s)...

Monitorando conexões dos participantes...

✓ Conectado: computador_2 — executa QuickSort (127.0.0.1:5301) (pestebubonika-26540)
✓ Conectado: computador_3 — executa multiplicação de matrizes (127.0.0.1:5302) (pestebubonika-6048)
Aguardando... 3/3 participantes conectados  (1s)

[OK] 3 participantes conectados. Prosseguindo...


=== Compilando: tests\teste_adicional1_paralelismo_3_computadores.minipar ===
[Fase 1] Analise Lexica e Sintatica...
[Fase 2] Analise Semantica e Tabela de Simbolos...
[Fase 3] Geracao de Codigo Intermediario (TAC - 3 enderecos)...
[TAC] Arquivo gerado: output\teste_adicional1_paralelismo_3_computadores.tac
[Fase 4] Transpilacao e Chamada do GCC (-O3)...
[CODEGEN] Compilando via g++ com -std=c++20 e -O3...
[SUCESSO] Executavel: output\teste_adicional1_paralelismo_3_computadores.exe

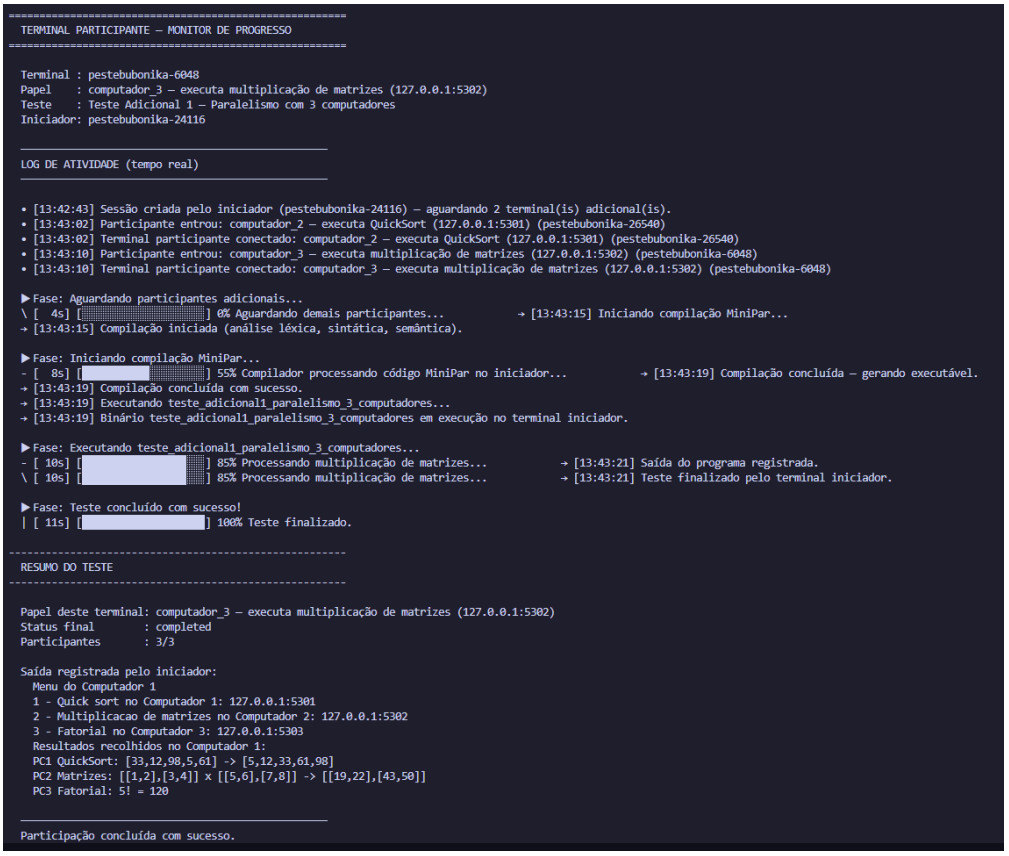
--- Executando teste_adicional1_paralelismo_3_computadores ---
==== Teste adicional 1: paralelismo real com 3 computadores ====
Menu do Computador 1
1 - Quick sort no Computador 1: 127.0.0.1:5301
2 - Multiplicacao de matrizes no Computador 2: 127.0.0.1:5302
3 - Fatorial no Computador 3: 127.0.0.1:5303
Resultados recolhidos no Computador 1:
PC1 QuickSort: [33,12,98,5,61] -> [5,12,33,61,98]
PC2 Matrizes: [[1,2],[3,4]] x [[5,6],[7,8]] -> [[19,22],[43,50]]
PC3 Fatorial: 5! = 120
```

Ln 368, Col 1

Terminal 2 - Participante



Terminal 3 - Participante



b)

```
=== Compilando: tests\teste_adicional2_tapete_sierpinski.minipar ===
[Fase 1] Analise Lexica e Sintatica...
[Fase 2] Analise Semantica e Tabela de Simbolos...
[Fase 3] Geracao de Codigo Intermediario (TAC - 3 enderecos)...
[TAC] Arquivo gerado: output\teste_adicional2_tapete_sierpinski.tac
[Fase 4] Transpilacao e Chamada do GCC (-O3)...
[CODEGEN] Compilando via g++ com -std=c++20 e -O3...
[SUCESSO] Executavel: output\teste_adicional2_tapete_sierpinski.exe

--- Executando teste_adicional2_tapete_sierpinski ---
==== Teste adicional 2: tapete de Sierpinski ====
[#, #, #, #, #, #, #, #, #]
[#,  , #, #,  , #, #,  , #]
[#, #, #, #, #, #, #, #, #]
[#, #, #,  ,  ,  , #, #, #]
[#,  , #,  ,  ,  , #,  , #]
[#, #, #,  ,  ,  , #, #, #]
[#, #, #, #, #, #, #, #, #]
[#,  , #, #,  , #, #,  , #]
[#, #, #, #, #, #, #, #, #]
-----------------------------------
```

11.3 Tabela de Estados dos Testes

Teste	Cenário coberto	Resultado observado
teste1_calculadora_cliente_s servidor	c_channel, send/receive, par e sockets localhost.	Servidor recebeu '+ 7 5' e retornou 'Resultado: 12'.
teste2_fatorial_fibonacci_par	Execução paralela de fatorial e Fibonacci.	Fatorial de 5 = 120; Fibonacci até 13.
teste3_neuronio	Classe, método, this, while e aprendizado simples.	Neurônio convergiu em 51 iterações.
teste4_rede_neural_xor	Funções, classe, backpropagation e exp.	Saídas aproximadas: 0.0068, 0.9864, 0.9892, 0.0140.
teste5_recomendacao	Listas, indexação, rede neural de recomendação.	Lista de produtos recomendados com probabilidades.
teste6_quicksort_classes	OO, recurso, listas e troca por índice.	Vetor ordenado: [-100, -40, -3, 10, 70, 80].
teste7_quicksort_iterativo_ta c	Funções, pilha manual e TAC específico.	Array ordenado: [5, 12, 33, 61, 98].

Teste	Cenário coberto	Resultado observado
teste_adicional1_paralelismo _3_computadores	Três canais localhost em bloco par.	Resultados de quicksort, matrizes e fatorial recolhidos.
teste_adicional2_tapete_sier pinski	Recursão, matriz e impressão de fractal.	Tapete 9x9 impresso.
teste_ascii_art_divertidapkrl	Matriz 32x64 e automático/fractal.	Arte ASCII renderizada.
teste_bnf_controles_input	for, do while, break, continue e input em função.	Controle OK: 7 2.

12. IMPLEMENTAÇÃO

A implementação do compilador MiniPar 2026.1 utilizou a linguagem Python 3 como ferramenta principal para a construção do *front-end* e do *middle-end* (Analisador Léxico, Sintático, Semântico e Gerador de TAC) devido à sua excelente manipulação de strings e tipagem dinâmica no manuseio da AST. Para o *back-end* e a geração do código alvo, utilizou-se a linguagem C++ (padrão C++20), que é compilada de forma nativa chamando o g++ com a flag de otimização de performance -O3. A simulação do paralelismo real de hardware foi mapeada diretamente para a biblioteca nativa std::thread, enquanto a comunicação de rede utilizou Sockets TCP convencionais em C++. O ambiente de desenvolvimento e a interface gráfica basearam-se no framework Flask (versão 3.0.0) para o servidor da IDE Web, auxiliado pelo CodeMirror para a renderização interativa do editor no navegador. O código-fonte implementado foi amplamente documentado; os arquivos Python, por exemplo, contêm comentários explicativos detalhando as regras de inferência de tipos e as produções BNF tratadas por cada função do analisador sintático.

12.1 Tecnologias utilizadas

Tecnologia	Uso
Python 3	Implementação do compilador, CLI, menu e backend Flask.
Flask 3.0.0	Servidor da IDE Web.
CodeMirror	Editor de código MiniPar, C++ e TAC no navegador.
C++20	Linguagem alvo gerada pelo compilador.
g++	Compilação do C + + gerado com otimização -O3.
std::thread	Execução de blocos para.
Sockets TCP	Implementação de c_channel, send e receive.
ReportLab	Geração deste relatório em PDF.

12.2 Fluxo de execução

O passo a passo da execução e validação dos testes ocorre através de um fluxo intuitivo na IDE Web construída para o compilador:

1. **Seleção do Código:** O usuário acessa a página inicial (IDE Web) onde a estrutura de arquivos da pasta tests/ é carregada na barra lateral. O usuário seleciona um arquivo .minipar (como o do Tapete de Sierpinski) e o código é carregado no editor de texto central.
2. **Disparo da Compilação:** Ao clicar no botão verde superior direito "Compile & Run", o *backend* Flask invoca o processo do compilador, que realiza sequencialmente a tokenização, a montagem da AST e a análise semântica.
3. **Geração de Artefatos:** Em questão de segundos, as abas auxiliares da interface são preenchidas. O painel central direito exibe o Código C++ final gerado para a

plataforma nativa, enquanto a aba "TAC Output" permite visualizar o código intermediário de três endereços destrinchado instrução a instrução.

4. **Execução e Resultado:** O binário otimizado .exe gerado pelo g++ é imediatamente executado pelo sistema operacional do servidor. Toda a saída padrão (stdout), incluindo os *prints* dos resultados matemáticos, matrizes fractais ou solicitações de *input*, é devolvida em tempo real e exibida no terminal de cor preta na porção inferior da tela.

(Alternativamente, via CLI interativa, o usuário digita no terminal o comando correspondente ao teste desejado, e o script menu.py imprime no console os logs de "Sucesso" de cada etapa morfológica e a saída da execução).

Exemplo de fluxo de execução:

```
python main.py tests/teste3_neuronio.minipar
-> Lexer.tokenize()
-> Parser.parse()
-> SemanticAnalyzer.analyze()
-> TACGenerator.generate()     # cria output/teste3_neuronio.tac
-> CppCodeGenerator.generate() # cria output/teste3_neuronio.cpp
-> g++ -std=c++20 -O3 ...      # cria output/teste3_neuronio.exe
-> menu.py ou app.py executa o binario e mostra a saida
```

12.3 Observações técnicas

O projeto prioriza simplicidade e legibilidade para fins didáticos. A inferência de tipos é propositalmente simples, usando **var** e **unknown** para permitir programas mais flexíveis. A geração C++ usa **auto** para reduzir a necessidade de um sistema de tipos completo. A comunicação por sockets já permite testes em localhost e pode ser estendida para máquinas distintas ajustando IP e porta.

Como melhoria futura, recomenda-se implementar recuperação de erros sintáticos sem encerrar o processo no primeiro erro, separar melhor tipos numéricos inteiros/reais e adicionar testes automatizados unitários por fase.

13. CONCLUSÃO

O **minipar-improvement** atende ao núcleo solicitado para o compilador MiniPar 2026.1: possui analisadores léxico, sintático e semântico, tabela de símbolos, orientação a objetos, paralelismo, canais de comunicação, TAC, geração de código C++ e interface web. A execução dos testes confirma o funcionamento integrado da cadeia de compilação e cobre os exemplos principais indicados pelo professor, além de cenários adicionais com matrizes, fractais e controles de fluxo.

14. LINKS ÚTEIS

1. Repositório do github: https://github.com/luizwhirl/minipar-improvement/tree/main
2. Especificações do Projeto (dado pelo professor): Especificações Projeto Compil-AB2
3. Apresentação em Slides pública: https://canva.link/t8lsru4tecpvjoz
4. Link desse documento: relatório-compiladoresOP3

REFERÊNCIAS

Referências baseadas nos materiais fornecidos pelo professor

1. AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. **Compiladores: princípios, técnicas e ferramentas**. 2. ed. São Paulo: Pearson Addison-Wesley, 2008.

2. SANTIAGO, Judson. **Compiladores: Introdução, Análise Semântica e Tabela de Símbolos**. Material de aula (Slides). [S.l.: s.n.], [20--].

3. UNIVERSIDADE FEDERAL DA PARAÍBA (UFPB). Departamento de Sistemas e Computação (DSC). **Parte 4: Tabela de Símbolos e Verificação Semântica de Tipos**.

4. DOMINGUEZ, Arturo Hernandez. **Elementos de Implementação Utilizando Componentes**. Material de apoio da disciplina de Compiladores. Maceió: Universidade Federal de Alagoas, Instituto de Computação, [202-].

5. DOMINGUEZ, Arturo Hernandez. **Exemplo programa MiniParOO converter para código C**. Material de apoio da disciplina de Compiladores. Maceió: Universidade Federal de Alagoas, Instituto de Computação, [202-].

6. MASSACHUSETTS INSTITUTE OF TECHNOLOGY (MIT). **Computation Structures: Structs e Memória**. Material de aula. Cambridge: MIT OpenCourseWare, [20--].

7. UNIVERSIDADE FEDERAL DE ALAGOAS (UFAL). Instituto de Computação. **Apostila LFeC II: Introdução à Compilação**. Material de apoio. Maceió: UFAL, [20--].

Referências adicionais (Tecnologias e Links)

1. HAVERBEKE, Marijn. **CodeMirror: In-browser code editor**. 2026. Disponível em: https://codemirror.net/. Acesso em: 15 mai. 2026.

2. ISO/IEC. **ISO/IEC 14882:2020: Programming Languages - C++**. Genebra: International Organization for Standardization, 2020. Disponível em: https://isocpp.org/std/the-standard. Acesso em: 28 mai. 2026.

3. LOUDEN, Kenneth C. **Compiladores: princípios e práticas**. São Paulo: Pioneira Thomson Learning, 2004.

4. PALLETS PROJECTS. **Flask Documentation (3.0.x)**. 2023. Disponível em: https://flask.palletsprojects.com/. Acesso em: 30 abril. 2026.

5. PYTHON SOFTWARE FOUNDATION. **The Python 3 Language Reference**. 2026. Disponível em: https://docs.python.org/3/reference/. Acesso em: 09 mai. 2026.